

Blood Bank Unit Allocation System

Design Document

1. Terminology & Formulae

- unit: one blood item represented as {"id": str, "group": str, "expiry": str, "status": str}. Status is Available, Issued, or Wasted. All statuses remain in stock.csv; only Available units count in the stock table.
- blood group: one of A+, A-, B+, B-, AB+, AB-, O+, O-.
- stock: the list of unit dictionaries in memory and in stock.csv. The persisted CSV uses the fields id, group, expiry, and status.
- request: one hospital request with id, hospital, hospital_type, items, urgency, and status. items is an ordered list of {"group": str, "quantity": int}; status is Pending for queued Mid/Low requests. High requests are allocated immediately and are not placed in the queue.
- request item: one requested group and quantity in a multi-item request. Item order is preserved during allocation, so units consumed by one item cannot be reused by a later item.
- compatibility: fixed mapping from donor stock group to allowed requested groups: O- -> all groups; O+ -> O+, A+, B+, AB+; A- -> A-, A+, AB-, AB+; A+ -> A+, AB+; B- -> B-, B+, AB-, AB+; B+ -> B+, AB+; AB- -> AB-, AB+; AB+ -> AB+ only.
- allocation result: a dictionary with request metadata, unit_ids, per-unit fees, unit details, item_results, total, and status. Status is Total (all requested quantities met), Partial (some units issued but some requested quantity remains unmet), or Rejected (no requested unit allocated).
- plan: proposed result from generate_plan: {"allocations": [...], "proposed_revenue": int}. It is a simulation and does not issue real stock. Confirming applies the result only after is_plan_valid succeeds.
- shelf-life window: Add Stock accepts a real YYYY-MM-DD date from today through today + 35 days, inclusive. A date before today or more than 35 days out is rejected. A unit with expiry equal to today is not expired until the date becomes earlier than today.
- persistence: storage.py reads and writes the inventory in stock.csv located beside storage.py. Inventory survives restarts. Pending requests, current_plan, request IDs, progress counts, and revenue do not survive a restart. At startup, persisted Wasted units are loaded into the current wasted report log.
- session log: {"fulfilled": 0, "rejected": 0, "revenue": 0, "wasted_units": [], "wasted_value": 0}. Partial allocations increment fulfilled; completely rejected requests increment rejected.

Fee formula (matches current code)

```
per_unit_fee = 500 + urgency_adjustment + donor_group_adjustment + private_surcharge
urgency_adjustment: High = 300, Mid = 100, Low = 0
donor_group_adjustment: Rh-negative (A-, B-, AB-, O-) = 150; Rh-positive = 0
private_surcharge: Private = 150; Govt = 0
transport_fee: Govt = 0; Private = 100 once per request when at least one unit is allocated
bill_total = sum(fees for units actually allocated) + transport_fee; a Rejected request total is 0
```

The implementation passes the selected unit's donor group to per_unit_fee. The current code therefore applies the Rh adjustment to the actual stock unit used, not necessarily the requested group. Private is a surcharge; the current code does not apply a Government discount.

2. Program Flow

1. Load stock.csv using storage.load_stock(). If the file does not exist, start with an empty inventory. Initialize pending_requests as an empty list, current_plan as None, a new session log, request_counter = 1, and the next unit counter from the highest saved U-number.
2. For every previously saved Wasted unit, add it to the current wasted log and add Rs 500 to the current wasted value. This reconstructs the wasted report from persisted inventory, not from a separate report file.
3. Read today from the computer's local date. Mark every Available unit with expiry earlier than today as Wasted, append newly wasted units to the wasted log, and attempt to save if stock changed.
4. Print the stock table (Available counts and nearest expiry for the eight groups), pending request count, and the seven-option menu.
5. Read the menu choice as stripped text. Reject non-digits and numbers outside 1–7 with an error, then repeat the loop.

6. Run the chosen action: Add stock; Submit request; Generate allocation plan; Confirm & issue plan; View progress report; View expired/wasted report; or Exit.
7. After a non-exit action, save stock if it differs from the last successfully saved snapshot. On the next loop, check expiries again before showing the menu.
8. On Exit, try to save once more, warn if stock changes remain unsaved, print Goodbye, and stop. Pending requests, current plan, progress counts, and revenue are not written to disk.

4. Data Structures

Data Structure	Used In	What It Does
Dictionary	Unit record	id, group, expiry, status for one blood unit.
List of dictionaries	stock	Full inventory, including Available, Issued, and Wasted units.
CSV file	stock.csv	Persistent inventory with columns id, group, expiry, status; written by storage.py.
Dictionary	Request	id, hospital, hospital_type, items, urgency, status.
List of dictionaries	Request items	Ordered {group, quantity} entries; multiple groups can be in one request.
Dictionary of lists	COMPATIBILITY	Maps each donor group to the requested groups it can supply.
Dictionary	Allocation result	Per-request status, selected unit IDs/details, line results, fees, and total.
Dictionary + list	Plan	Proposed allocations and proposed_revenue; plan is held in memory only.
List of dictionaries	pending_requests	Queued Pending Mid/Low requests, in received order before urgency grouping.
Dictionary + list	session_log	Fulfilled/rejected counts, revenue, wasted units, and wasted value for this run.
Integer	unit_counter	Next U-number, recalculated from persisted stock at startup and incremented for each accepted stock unit.
Integer	request_counter	Next R-number; starts at 1 each run because requests are not persisted.
List snapshot	last_saved	Inventory snapshot last successfully written; used to decide whether another save is needed and to retry after a failed save.

5. Function Signatures

The Python functions do not declare type annotations in source; the signatures below list the implemented function names and parameters, with return behaviour described in the second column.

blood_bank_logic.py - allocation and validation

Function	Purpose / Return Behaviour
new_session_log()	Return a fresh session totals dictionary with zero counts/revenue and an empty wasted list.
is_valid_group(group)	True only for one of the eight supported groups.
is_valid_quantity(quantity)	True for digit-only text that converts to an integer greater than zero.
is_valid_urgency(urgency)	True when value is exactly High, Mid, or Low; main.py normalizes typed values first.
is_valid_hospital_type(hospital_type)	True when value is exactly Private or Govt; main.py normalizes typed values first.
is_valid_date(text)	True when length is 10 and datetime.strptime accepts YYYY-MM-DD as a real calendar date.
is_expired(expiry, today)	True when expiry is strictly earlier than today. Inputs are expected to be valid ISO dates.

Function	Purpose / Return Behaviour
is_within_shelf_life(expiry, today)	True when expiry is on or before today + 35 days; this function does not itself reject past dates.
add_unit(stock, unit_id, group, expiry, today)	Return a new list with an Available unit appended if expiry is not past and is within 35 days; otherwise return the original stock unchanged.
expire_units(stock, today)	Return (new_stock, newly_wasted_units), converting only expired Available units to Wasted.
compatible_available_units(stock, request_group)	Return all compatible Available units sorted by expiry ascending, then unit ID ascending.
per_unit_fee(group, urgency, hospital_type)	Compute Rs 500 base + urgency adjustment + Rh-negative group adjustment + Rs 150 Private surcharge.
allocate_request(stock, request)	Simulate allocation across each request item without mutating the caller's inventory; returns Total, Partial, or Rejected result.
issue_units(stock, unit_ids)	Return a new stock list with matching IDs marked Issued.
sort_for_plan(requests)	Return only Pending Mid/Low requests, Mid first then Low, preserving received order within each urgency.
generate_plan(stock, requests)	Simulate all eligible requests with a working stock list; return allocations and proposed_revenue without changing real stock.
is_plan_valid(stock, plan)	True if every unit ID proposed in Total/Partial allocations still exists with status Available.
apply_plan(stock, plan, session_log)	Assuming prior validation, issue units and update a copied session log; Total/Partial increment fulfilled and revenue, Rejected increments rejected.
wasted_report(session_log)	Return wasted-unit count and wasted value.
progress_report(session_log)	Return handled, fulfilled, rejected, and revenue; handled equals fulfilled + rejected.

main.py - user interface and control flow

Function	Purpose / Return Behaviour
render_stock_table(stock)	Return an eight-group table counting Available units and showing the nearest expiry or "-".
render_menu()	Return the seven menu options as printable text.
read_add_stock_input()	Prompt for comma-separated group/date pairs; each pair must provide at least two whitespace-separated tokens. Extra tokens are ignored by the current parser.
read_request_input()	Prompt for hospital, hospital type, urgency, and raw multi-item input text.
print_bill(allocation)	Print unit details and bill for Total/Partial; print a completely-rejected message for Rejected.
print_plan(plan)	Print every planned request and its outcome, proposed revenue, and confirmation instruction.
print_progress_report(report)	Print handled, fulfilled, rejected, and current-run revenue.
print_wasted_report(report, stock)	Print every inventory unit marked Wasted and the count/value summary.
save_if_changed(stock, last_saved)	Save only when stock differs from the last successfully saved snapshot; return the new snapshot on success or old snapshot on failure.
main()	Load state, expire stock, display menu, validate inputs, perform actions, retry saves, and exit.

storage.py - disk access

Function	Purpose / Return Behaviour
save_stock(stock, path=None)	Write CSV header and rows to path + .tmp and use os.replace to replace the target. Return True on success; clean up the temp file, print an error, and return False for PermissionError/OSError.
load_stock(path=None)	Read CSV using utf-8-sig and DictReader; return an empty list if the file does not exist. Other malformed-data cases are not comprehensively validated.
next_unit_counter(stock)	Return one more than the highest numeric suffix in saved unit IDs; return 1 for an empty list.
_remove_quietly(path)	Try to delete the temporary file and ignore deletion errors.

6. Function-Level Algorithm

blood_bank_logic.py

new_session_log()

- Create fulfilled = 0, rejected = 0, revenue = 0, wasted_units = [], and wasted_value = 0; return the dictionary.

Validation helpers

- is_valid_group checks membership in the fixed group list.
- is_valid_quantity calls isdigit() and checks the integer is greater than zero; the CLI passes input text.
- is_valid_urgency and is_valid_hospital_type compare against their exact supported values after main.py has normalized user input.
- is_valid_date rejects text whose length is not 10, then attempts datetime.strptime(text, "%Y-%m-%d"); ValueError means invalid.
- is_expired parses expiry and today and returns true only when expiry date < today date.
- is_within_shelf_life computes today + 35 days and accepts expiry <= cutoff. Add-stock validation calls is_expired separately to reject past dates.

add_unit(stock, unit_id, group, expiry, today)

- If expiry is in the past, return the original stock. If it is more than 35 days ahead, also return the original stock.
- Otherwise copy the list, append {id, group, expiry, status: Available}, and return the new list. The CLI validates group/date before calling it.

expire_units(stock, today)

- Loop through stock. If a unit is Available and expiry < today, copy it with status Wasted and add it to both the updated stock and the newly wasted list.
- Leave Issued units, already-Wasted units, and units expiring today unchanged. Return (updated_stock, newly_wasted_list).

compatible_available_units(stock, request_group)

- Keep units whose status is Available and whose donor group's COMPATIBILITY list includes request_group.
- Sort by (expiry, id) ascending and return the sorted list.

per_unit_fee(group, urgency, hospital_type)

- Start at Rs 500. Add Rs 300 for High, Rs 100 for Mid, or Rs 0 for Low.
- Add Rs 150 if the selected stock unit's group is Rh-negative. Add Rs 150 when hospital_type is Private; add nothing for Govt. Return the fee.

allocate_request(stock, request)

- Create a working copy of stock and empty lists for unit IDs, fees, unit details, and item results.
- Process request items in input order. For each item, get compatible Available units from the current working stock.
- If there are none, add an item result with allocated_qty 0 and status Rejected. Otherwise choose min(quantity, number of compatible units); status is Total for a fully satisfied item or Partial for a shortage.
- Calculate the fee for each selected unit from that unit's donor group, record its details, and mark it Issued in the working copy so a later item cannot reuse it.

- Overall status is Rejected if no units were selected, Total if all requested quantities were met, otherwise Partial. Rejected total is Rs 0. For Total/Partial, total = sum(fees) + transport_fee, where transport_fee is Rs 0 for Govt and Rs 100 for Private. The input stock is not modified.

issue_units(stock, unit_ids)

- Create and return a new list. For each unit whose ID is included, return a copy with status Issued; leave other records unchanged.

sort_for_plan(requests)

- Collect only requests with status Pending and urgency Mid, preserving their input order. Collect Pending Low requests likewise.
- Return the Mid list followed by the Low list. High requests are never included in the batch plan.

generate_plan(stock, requests)

- Set working_stock to the input stock, allocations to an empty list, and proposed_revenue to zero.
- For every request in sort_for_plan order, call allocate_request and append the result. For Total/Partial results, mark proposed IDs Issued only in working_stock and add the bill total to proposed_revenue.
- Return {allocations, proposed_revenue}; the real stock remains unchanged.

is_plan_valid(stock, plan)

- For every Total/Partial allocation, confirm that every proposed unit ID still exists in stock with status Available.
- Return false at the first unavailable/missing proposed unit; otherwise return true. Rejected allocations do not require units.

apply_plan(stock, plan, session_log)

- This function assumes main.py already called is_plan_valid.
- For every Total/Partial allocation, mark proposed units Issued, add the bill to revenue, and increment fulfilled. For every Rejected allocation, increment rejected.
- Copy all previous totals and wasted-log fields before updates; return updated stock and updated session log.

wasted_report(session_log) and progress_report(session_log)

- wasted_report returns count = len(wasted_units) and the stored wasted_value.
- progress_report returns handled = fulfilled + rejected, both counters, and revenue. Partial results contribute to fulfilled through the caller/apply_plan.

main.py

Startup and main loop

- Load stock from storage.load_stock; create empty pending_requests, current_plan = None, fresh session_log, next unit ID, request_counter = 1, and last_saved snapshot.
- Repopulate the session wasted log with each unit already marked Wasted in stock.csv, adding Rs 500 loss per unit.
- On each loop, calculate local today; expire and log Available units whose expiry is before today; call save_if_changed; then print stock table, pending count, and menu.
- Strip the menu entry, reject anything not all digits or outside 1–7, and restart the loop.

Option 1: Add stock

- Read comma-separated pairs. Empty pieces are skipped; pairs with fewer than two tokens are reported and skipped; later tokens after group/date are ignored by the current parser.
- Normalize the group to uppercase; validate group, date format, past-date status, and 35-day cutoff in that order. For each accepted pair, assign the next U-number, call add_unit, increment the counter, and print Added.
- After the action, save inventory if changed.

Option 2: Submit request

- Read hospital name, hospital type, urgency, and raw item lines. Reject blank hospital names; normalize type/urgency case-insensitively.
- Split request items on commas. Skip empty pieces. Require at least two tokens; normalize group to uppercase; validate group and positive integer quantity. Any invalid item rejects the entire request before request ID assignment. Extra tokens after quantity are ignored by the current parser.

- If there are no valid item lines, print Status: Rejected and create no request. Otherwise create one request with an ordered items list.
- For High, call `allocate_request` immediately. Total/Partial results issue selected units and increment fulfilled/revenue; Rejected increments rejected. For Mid/Low, append the request as Pending to `pending_requests`.

Option 3: Generate allocation plan

- Check for at least one Pending Mid/Low request. If none exist, print the no-pending message and set `current_plan` to None.
- Otherwise call `generate_plan(persisted stock, pending_requests)`, save the returned plan in memory, and print proposed outcomes and revenue. No disk state is changed by planning itself.

Option 4: Confirm & issue plan

- If `current_plan` is None, print an error. Otherwise validate with `is_plan_valid`.
- If stale, print the stale-plan message and clear `current_plan`. If valid, call `apply_plan`, remove every request ID included in its allocations from `pending_requests`, print confirmation/revenue, and clear `current_plan`.

Options 5-7 and save points

- Option 5 prints `progress_report(session_log)`; option 6 prints all stock entries whose status is Wasted and the wasted summary.
- Option 7 attempts final save, warns if stock still differs from `last_saved`, prints Goodbye, and breaks.
- After any non-exit action, `save_if_changed` writes inventory only when it differs from `last_saved`. A failed write leaves `last_saved` unchanged so the next save point retries.

storage.py

`save_stock(stock, path=None)`

- Use `STOCK_FILE`, which is built from the `storage.py` folder, when no explicit path is provided.
- Write the standard CSV header and rows to `path + .tmp`, then call `os.replace` to move the complete temporary file over the target.
- On `PermissionError` or `OSError`, try to remove the temporary file, print a clear error, and return False. On success, optionally print the saved path and return True.

`load_stock(path=None)`

- Open the target with encoding `utf-8-sig` and `csv.DictReader` so a UTF-8 BOM from spreadsheet software is tolerated.
- Convert rows to dictionaries. If the target does not exist, return an empty list. The current code does not validate every malformed row or column before use.

`next_unit_counter(stock)`

- Read the numeric suffix after the leading U of each saved unit ID, find the highest, and return `highest + 1`; return 1 for empty stock.
- This assumes valid generated IDs such as U001. Malformed IDs are not guarded against.

`save_if_changed(stock, last_saved)`

- If stock equals `last_saved`, return `last_saved` without writing.
- If `save_stock` succeeds, return stock as the new saved snapshot. If it fails, return the old snapshot so the difference is retried at a later save point.

7. Edge Cases & Failure Handling

Scenario	Implemented Handling / Limitation
Date input	Invalid shape, impossible date, a date in the past, or a date > 35 days ahead is rejected before <code>add_unit</code> is called.
Date boundary	Today and exactly today + 35 days are valid. Expiration uses <code>expiry < today</code> , so <code>expiry today</code> remains Available.
Mixed stock entry	A bad group or malformed pair is skipped without blocking other comma-separated pairs.
Multi-item request validation	If one non-empty item line fails syntax/group/quantity validation, the whole request is rejected before any allocation.

Scenario	Implemented Handling / Limitation
Partial availability	Each item receives as many compatible units as possible. The request can therefore be Partial; chosen units are billed and issued at commit/immediate handling.
No availability	If no units are allocated to any item, status is Rejected, total is 0, no transportation charge is added, and no units are issued.
Duplicate item groups	Lines are processed in the entered order; the working stock is updated after each line, so an ID cannot be selected twice within one request.
Stale plan	Confirmation checks the availability of all proposed IDs. One missing, expired, or Issued ID invalidates the whole plan and clears current_plan.
Missing stock.csv	The application starts from an empty list; the file is created at the first successful save caused by a stock change.
Locked/unwritable stock.csv	save_stock returns False and prints an error instead of exiting. save_if_changed retains the previous snapshot for retries; the exit path warns if data remains unsaved.
Restart boundaries	Inventory statuses persist. Pending requests, plans, request counter, progress totals, and revenue are recreated on each run. Wasted status is persistent and its report is reconstructed from inventory.
Malformed stock.csv	Current code expects four columns and U-number IDs; corrupted rows/IDs are not comprehensively validated and may cause errors during startup or ID generation.
Extra tokens in user input	Current parsers use the first two tokens for stock pairs and first two tokens for each request item; later tokens are ignored. Users should enter only the documented fields.
Transportation billing	For a Total or Partial allocation, charge Govt Rs 0 transportation or Private Rs 100 once per request. A Rejected request total is Rs 0 with no transportation. The Private per-unit surcharge of Rs 150 is separate from transportation.

8. Test Coverage

The supplied test_blood_bank_logic.py contains 20 scenario-based unit tests, and all 20 pass against the current code. The table below lists those actual test scenarios. These tests do not directly exercise the full CLI menu/prompt flow, and they do not directly assert the changed Govt-versus-Private transportation amount or every fee-formula combination; add integration/fee tests for those behaviours when automated coverage is required.

Test	Scenario / Function	Expected Result
TC01 / FR1	Saved stock round-trip (storage.save_stock / load_stock)	One saved Available unit loads back with status Available.
TC02 / FR2	Add valid O+ unit inside 35-day window (add_unit)	One unit is appended with Available status.
TC03 / FR2	Unsupported group C+ (is_valid_group)	Returns False.
TC04 / FR2	Past expiry date (is_expired)	2025-01-01 is expired relative to 2026-10-08.
TC05 / FR2	Impossible date 2026-02-30 (is_valid_date)	Returns False.
TC06 / FR6	Expiry exactly 35 days ahead (is_within_shelf_life)	Returns True; cutoff is inclusive.
TC07 / FR6	Expiry 36 days ahead (is_within_shelf_life)	Returns False.
TC08 / FR3	High request with enough A+ stock (allocate_request)	Status Total; one unit ID selected.
TC09 / FR3	High request with partial A+ stock (allocate_request)	Status Partial; one available unit selected.
TC10 / FR3	High request with empty stock (allocate_request)	Status Rejected; no unit IDs selected.
TC11 / FR3	High request containing A+ and B+ items (allocate_request)	Status Total; two units selected.
TC12 / FR4	Low request followed by Mid request competing for one unit (generate_plan)	Mid is processed first and is Total; Low is Rejected.

Test	Scenario / Function	Expected Result
TC13 / FR4	Two Mid requests compete for one unit (generate_plan)	First request is Total; second request is Rejected.
TC14 / FR5	Apply a valid Total plan (apply_plan)	Unit becomes Issued; revenue increases by the provided Rs 600 total.
TC15 / FR5	Validate plan after proposed unit is already Issued (is_plan_valid)	Returns False.
TC16 / FR7	Progress totals for 3 fulfilled and 2 rejected (progress_report)	Handled 5, fulfilled 3, rejected 2.
TC17 / FR8	Two expired Available units and wasted report (expire_units / wasted_report)	Both are Wasted; count 2 and value Rs 1000.
TC18 / FR10	Save and reload stock.csv (storage)	Unit ID U001 is restored.
TC19 / FR10	Load a missing stock.csv (storage.load_stock)	Returns an empty list.
TC20 / FR10	Simulate locked stock.csv during replace (storage.save_stock)	Save returns False and prior U001 data stays intact.

Recommended additional integration checks: full main() startup and CLI input parsing; invalid multi-item request handling; billing assertions for the per-unit formula and Govt (Rs 0) versus Private (Rs 100) transportation; a stale plan after a High request issues one of its proposed units; restart boundaries and unit-ID continuation; and recovery or clear diagnostics for malformed stock.csv data.